



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico

El subtitulo adecuado

September 2, 2026

Algoritmos y Estructuras de Datos

Java con Mates

Integrante	LU	Correo electrónico
Ayala, Ignacio	196/25	ayalanacho11@gmail.com
Della Bonzana, Lara	563/25	dellabonzanalara@gmail.com
Morales, Santiago	481/25	santumorales11@gmail.com
Vannelli, Tiziana	945/25	tizivannelli@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (+54 +11) 4576-3300

<http://www.exactas.uba.ar>

1 Macros para escribir en LaTeX

Para facilitar la escritura del trabajo práctico en LaTeX, les proveemos de varias macros que les permitirán escribir especificaciones en lógica de primer orden. Por ejemplo, podrán hacer uso de comandos que definen los conectores lógicos: $\wedge, \vee, \rightarrow$ y también los operadores de la lógica trivaluada $\wedge_L, \vee_L, \rightarrow_L$.

Además, disponen de comandos para cuantificadores: $(\forall x : \mathbb{Z}) (x \geq 0 \vee x < 0)$, $(\exists x : \mathbb{Z}) (x \geq 0)$. A estos comandos se les puede agregar un **Largo** al final para que ocupen múltiples renglones.

2 Ejemplos de uso de las macros

```
TAD EjemploDeTAD {
  obs secuenciaDeCaracteres : seq⟨char⟩
  obs conjuntoDeTuplas : conj⟨⟨nombre : seq⟨char⟩, apellido : seq⟨char⟩⟩⟩

  proc crearTAD(in caracteres : seq⟨char⟩, in secuenciaDeNombres : seq⟨⟨nombre : seq⟨char⟩, apellido :
  seq⟨char⟩⟩⟩) : EjemploDeTAD {
    requiere { predicadoUnilinea(caracteres) }
    asegura { res.secuenciaDeCaracteres = caracteres }
    asegura {
      |res.conjuntoDeTuplas| = |secuenciaDeNombres|  $\wedge$ 
      predicadoMultilinea(secuenciaDeNombres, res.conjuntoDeTuplas)
    }
  }
  pred predicadoMultilinea(s : seq⟨char⟩, conjunto : conj⟨⟨seq⟨char⟩, seq⟨char⟩⟩⟩) {
     $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i] \in c)$ 
  }
  pred predicadoUnilínea(s : seq⟨char⟩) { |s| > 0 }
  aux unAuxiliar(k :  $\mathbb{Z}$ , l :  $\mathbb{Z}$ ) :  $\mathbb{Z} = |k - l|$ 
}
```

Usuario ES $\langle id : \mathbb{Z}, millasTotales : \mathbb{Z}, millasDisponibles : \mathbb{Z}, historial : seq\langle \rangle, categoria : Categoria \rangle$
 Promocion ES $\langle nombre : char, factor : \mathbb{Z}, estado : bool \rangle$
 Categoria ES char

TAD AirMiles {

```

    obs usuarios : conj⟨Usuario⟩
    obs promociones : conj⟨Promocion⟩
    obs promocionesConTopeActivas : dict⟨Promocion, ⟨ℤ, ℤ⟩⟩

    proc crearSistema() : AirMiles {
      requiere { True }
      asegura { (∃p : Promocion) (p ∈ res.promociones ∧ p.nombre = ' NoPromo' ∧
        p.factor = 1 ∧ p.estado = True) }
      asegura { res.usuarios = ∅ }
      asegura { #res.promociones = 1 }
    }

    proc registrarUsuario(inout a : AirMiles, in id : ℤ) {
      requiere { !esUsuarioRegistrado(id, a) ∧ a = A0 }
      asegura { #a.usuarios = #A0.usuarios + 1 }
      asegura { (∀u : Usuario) (esUsuarioRegistrado(u.id, A0) → esUsuarioRegistrado(u.id, a)) }
      asegura { (∃u : Usuario) (!esUsuarioRegistrado(u.id, A0) ∧ esUsuarioRegistrado(u.id, a) ∧
        u.id = id ∧ u.millasDisponibles = 0 ∧ u.millasTotales = 0 ∧ u.historial = [] ∧
        u.categoria = ' Bronce') }
      asegura { a.promociones = A0.promociones }
    }

    proc registrarUsuarioConReferidor(in idNuevoUsuario : ℤ, in referidor : Usuario, inout a :
    AirMiles) {
      requiere { esUsuarioRegistrado(referidor.id, a) ∧ a = A0 ∧ !esUsuarioRegistrado(idNuevoUsuario, a) }
      asegura {
        (∃u : Usuario) (esUsuarioRegistrado(u.id, a) ∧ u.id = idNuevoUsuario ∧ u.millasTotales =
        500 ∧ u.millasDisponibles = 500 ∧ u.historial = [] ∧ u.categoria = Bronce)
      }
      asegura { elRestoDeUsuariosSeMantienenIgual(referidor, A0, a) }
      asegura { #a.usuarios = #A0.usuarios + 1 }
      asegura {
        (∃u : Usuario) (esUsuarioRegistrado(u.id, a) ∧ tienenMismoId(u, referidor) ∧
        u.millasDisponibles = referidor.millasDisponibles + 500 ∧ u.millasTotales =
        referidor.millasTotales + 500 ∧ u.historial = referidor.historial ++ [] ∧ u.categoria =
        queCategoriaEs(u.millasTotales, a))
      }
      asegura { a.promociones = A0.promociones }
    }

    proc acumularMillas(in usuario : Usuario, in distancia : ℤ, in nombrePromo : char, inout a :
    AirMiles) {
      requiere {
        esUsuarioRegistrado(usuario.id, a) ∧ a = A0 ∧ distancia ≥ 0 ∧ esPromoActiva(nombrePromo, a)
      }
    }
  
```

```

    }
    asegura { h }
}
proc canjearMillas(in u : Usuario, in cantMillas :  $\mathbb{Z}$ , in premio : T, inout a : AirMiles) {
    requiere {
        cantMillas > 0  $\wedge$  esUsuarioRegistrado(u.id, a)  $\wedge$  a = A0  $\wedge$  tieneSaldoSuficiente(u, cantMillas)  $\wedge$ 
        NoSuperaMillasTotales(u, cantMillas)
    }
    asegura { #A0.usuarios = #a.usuarios }
    asegura {
        ( $\exists u_0 : \text{Usuario}$ ) (esUsuarioRegistrado(u0.id, a)  $\wedge$  tienenMismoId(u0, u)  $\wedge$  u0.millasDisponibles =
        u.millasDisponibles - cantMillas  $\wedge$  tienenMismasMillasTotales(u, u0)  $\wedge$  u0.historial =
        u.historial + +[]  $\wedge$  tienenMismaCategoria(u, u0))
    }
    asegura { A0.promociones = a.promociones }
    asegura { elRestoDeUsuariosSeMantienenIgual(u, A0, a) }
}
proc transferirMillas(in cantMillas :  $\mathbb{Z}$ , in usuarioOrigen : Usuario, in usuarioDestino :
Usuario, inout a : AirMiles) {
    requiere {
        esUsuarioRegistrado(usuarioDestino.id, a)  $\wedge$  esUsuarioRegistrado(usuarioOrigen.id, a)  $\wedge$ 
        cantMillas  $\geq$  0  $\wedge$  tieneSaldoSuficiente(usuarioOrigen, cantMillas)  $\wedge$  usuarioDestino  $\neq$ 
        usuarioOrigen  $\wedge$  a = A0  $\wedge$  NoSuperaMillasTotales(usuarioDestino, cantMillas)
    }
    asegura { ( $\forall u : \text{Usuario}$ ) (esUsuarioRegistrado(u.id, A0)  $\wedge$  u  $\neq$  usuarioOrigen  $\wedge$  u  $\neq$ 
        usuarioDestino  $\rightarrow$  esUsuarioRegistrado(u.id, a)) }
    asegura { #A0.usuarios = #a.usuarios }
    asegura {
        ( $\exists u_0 : \text{Usuario}$ ) (esUsuarioRegistrado(u0.id, a)  $\wedge$  tienenMismoId(u0, usuarioOrigen)  $\wedge$ 
        u0.millasDisponibles = usuarioOrigen.millasDisponibles - cantMillas  $\wedge$  tienenMismasMillas
        u0.historial = usuarioOrigen.historial + +[]  $\wedge$  tienenMismaCategoria(u0, usuarioOrigen))
    }
    asegura {
        ( $\exists u_0 : \text{Usuario}$ ) (esUsuarioRegistrado(u0, a)  $\wedge$  tienenMismoId(u0, usuarioDestino)  $\wedge$ 
        u0.millasDisponibles = usuarioDestino.millasDisponibles + cantMillas  $\wedge$  tienenMismasMilla
        u0.historial = usuarioDestino.historial + +[]  $\wedge$  tienenMismaCategoria(u0, usuarioDestino))
    }
}
proc consultarSaldo(in u : Usuario, in a : AirMiles) :  $\mathbb{Z}$  {
    requiere { esUsuarioRegistrado(u.id, a) }
    asegura { res = u.millasDisponibles }
}
proc consultarCategoria(in u : Usuario, in a : AirMiles) : Categoria {
    requiere { esUsuarioRegistrado(u.id, a) }
    asegura { res = u.categoria }
}
proc rankingMillas(tipos) : AirMiles {
    requiere { True }
    asegura { algo }
}

```

```

}
proc resumenTransacciones(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
proc iniciarPromocion(in nombre : char, in factor :  $\mathbb{Z}$ , inout a : AirMiles) {
  requiere { factor  $\geq 1 \wedge a = A0$  }
  asegura { ( $\exists p : Promocion$ ) ( $esPromocionDelSistema(p, a) \wedge p.nombre = nombre \wedge$ 
     $p.factor = factor \wedge p.estado = True$ ) }
  asegura {  $elRestoDePromocionesSeMantienenIgual(nombre, A0, a) \wedge elRestoDePromocionesSeM$ 
  }
  asegura {  $a.usuarios = A0.usuarios$  }
}
Observacion: Usamos el predicado dos veces pero con distinto orden para afirmar que
toda promocion esta en un sistema si y solo si esta en el otro
proc iniciarPromocionConTope(in nombre : char, in factor :  $\mathbb{Z}$ , in tope :  $\mathbb{Z}$ , inout a :
AirMiles) : AirMiles {
  requiere {  $a = A0 \wedge factor \geq 0 \wedge tope \geq 0$  }
  asegura {  $elRestoDePromocionesSeMantienenIgual(nombre, A0, a) \wedge elRestoDePromocionesSeM$ 
  }
}
proc finalizarPromocion(in nombre : char, inout a : AirMiles) {
  requiere {  $esPromoActiva(nombre, a) \wedge a = A0 \wedge nombre \neq NoPromo$  }
  asegura { ( $\exists p : Promocion$ ) ( $p.nombre = nombre \wedge p.estado = False \wedge esPromocionDelSistema(p, a)$ )
  }
  asegura {  $elRestoDePromocionesSeMantienenIgual(nombre, A0, a)$  }
  asegura {  $\#a.promociones = \#A0.promociones$  }
  asegura {  $a.usuarios = A0.usuarios$  }
}
proc elQueTieneMasMillas(in a : AirMiles) :  $\mathbb{Z}$  {
  requiere { True }
  asegura { ( $\exists u : Usuario$ ) ( $esUsuarioRegistrado(u.id, a) \wedge u.id = res \wedge tieneMaximasMillas(u, a)$ )
  }
}
proc elMasCanjeador(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
proc paresTransferidoresExclusivos(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
aux queCategoriaEs(millas :  $\mathbb{Z}$ , a : AirMiles) : Categoria = IfThenElse( $millas \leq 10000$ , Bronce,
IfThenElse( $millas \leq 50000$ , Plata, IfThenElse( $millas \leq 1000000$ , Oro, Platino)))

aux factorDeCategoria(categoria : Categoria) :  $\mathbb{Z}$  = IfThenElse( $categoria = Bronce$ , 1, IfThenElse( $cate$ 
Plata, 1.5, IfThenElse( $categoria = Oro$ , 2, 0)))
aux calculoMillas(distancia :  $\mathbb{Z}$ , factorCategoria :  $\mathbb{Z}$ , factorPromocion :  $\mathbb{Z}$ ) :  $\mathbb{Z}$  =
distancia * factorCategoria * factorPromocion

```

```

pred tieneMaximasMillas(u : Usuario, a : AirMiles) { ( $\forall u_0$  : Usuario) ( $u_0 \in a.usuarios \wedge u_0 \neq u \rightarrow u.millasTotales \geq u_0.millasTotales$ ) }
pred esUsuarioRegistrado(id :  $\mathbb{Z}$ , a : AirMiles) { ( $\exists u_0$  : Usuario) ( $u_0 \in a.usuarios \wedge_L u_0.id = id$ ) }
pred esPromoActiva(nombre : char, a : AirMiles) { ( $(\exists p$  : Promocion) ( $p \in a.promociones \wedge_L p.nombre = nombre \wedge p.estado = True$ )) }

pred elRestoDeUsuariosSeMantienenIgual(usuario : Usuario, A0 : AirMiles, a : AirMiles)
{ ( $\forall u$  : Usuario) ( $u \in A0.usuarios \wedge u \neq usuario \rightarrow_L u \in a.usuarios$ ) }
pred elRestoDePromocionesSeMantienenIgual(nombrePromocion : char, A0 : AirMiles, a : AirMiles)
{ ( $\forall p$  : Promocion) ( $p \in A0.promociones \wedge p.nombre \neq nombrePromocion \rightarrow p \in a.promociones$ ) }
pred esPromocionDelSistema(promo : Promocion, a : AirMiles) {  $p \in a.promociones$  }
pred tienenMismoId(u0 : Usuario, u1 : Usuario) {  $u_0.id = u_1.id$  }
pred tienenMismasMillasDisponibles(u0 : Usuario, u1 : Usuario) {  $u_0.millasDisponibles = u_1.millasDisponibles$  }
pred tienenMismasMillasTotales(u0 : Usuario, u1 : Usuario) {  $u_0.millasTotales = u_1.Totales$  }
pred tienenMismaCategoria(u0 : Usuario, u1 : Usuario) {  $u_0.categoria = u_1.categoria$  }
pred tienenMismoHistorial(u0 : Usuario, u1 : Usuario) {  $u_0.historial = u_1.historial$  }
pred tieneSaldoSuficiente(u : Usuario, cantMillas :  $\mathbb{Z}$ ) {  $u.millasDisponibles \geq cantMillas$  }
pred NoSuperaMillasTotales(u : Usuario, cantMillas :  $\mathbb{Z}$ ) {  $u.millasTotales \geq u.millasDisponibles + cantMillas$  }
}

```